

Hanxi "Gary" Chen

hanxic@cs.cornell.edu | www.hanxic.com | github.com/hanxic | linkedin.com/in/hanxic

EDUCATION & AWARDS

Cornell University

Ph.D. in Computer Science, Programming Languages
Advisors: Prof. Alexandra Silva, Prof. Andrew C. Myers

Ithaca, NY
2025 – Present

University of Pennsylvania, Penn Engineering & The Wharton School

M.S.E. in Computer and Information Science
B.S.E. in Computer Science, Mathematics minor
B.S. in Economics

Philadelphia, PA
May 2025

GPA: 3.97/4.00, *Summa Cum Laude*

Advisors: Prof. Steve Zdancewic, Prof. Sanjeev Khanna

Awards: MCIT TA Award • CIS Faculty Appreciation Award • CIS Senior Thesis • Wharton Research Scholars

SELECTED PUBLICATIONS

Calvin Beck, **Hanxi Chen**, Steve Zdancewic. [Vellvm: Formalizing the Informal](#). CoqPL'25, 2025.

Calvin Beck, **Hanxi Chen**, Steve Zdancewic. [Vellvm: Formalizing the Informal LLVM \(An Experience Report\)](#). NFM'25, 2025.

Calvin Beck, Irene Yoon, **Hanxi Chen**, Yannick Zakowski, Steve Zdancewic. [A Two-Phase Infinite/Finite Low-Level Memory Model: Reconciling Integer-Pointer Casts, Finite Space, and undef at the LLVM IR Level of Abstraction](#). ICFP'24, 2024.

Hanxi Chen, Noam Zilberstein, Andrew C. Myers, Alexandra Silva. [Oblivious Probabilistic Outcome Logic: Verifying Probabilistic Programs with an Oblivious Adversary](#). arXiv preprint, 2026.

RELEVANT RESEARCH EXPERIENCE

Research Assistant, *Cornell PL Group*, advised by Prof. Alexandra Silva and Prof. Andrew C. Myers
Cornell University, Computer Science

Sep 2025 – Present

- Working toward end-to-end security guarantees – reasoning about adversaries in probabilistic and concurrent systems, and carrying those guarantees down to executed code through semantics-preserving compilation.
- Developed opOL, a program logic for probabilistic nondeterministic programs under oblivious adversaries, and used it to verify randomized caching and leader-election protocols; the metatheory and case studies are fully mechanized in Lean 4.
- Led the Lean mechanization of Type-Confusion Security and proved its type safety, enforcing safety in decentralized systems whose untrusted components misrepresent dynamic types and induce confused-deputy attacks.

Research Mentor, *Cornell Math+AI*, advised by Prof. Daniel Halpern-Leistner

Jan 2026 – Present

Cornell University, Mathematics

- Mentored the Jan–Aug 2026 iteration of an autoformalization project, producing a proof-quality evaluation platform and LaTeX-to-Lean translation tools with interactive dependency graphs; now also directing a newer project on large language models for program reasoning.

Research Assistant, *PLClub*, advised by Prof. Steve Zdancewic

Jun 2021 – Aug 2025

University of Pennsylvania, Computer and Information Science

- Co-authored the ICFP 2024 Vellvm memory-model paper – designed a low-level memory model for LLVM optimization reasoning and verified its serialization lemmas in Rocq.
- Built a Rocq/OCaml differential-testing framework validating Vellvm against real LLVM implementations, with QuickChick generators for UB-free programs; found previously unreported LLVM bugs.
- Developed Rocq infrastructure for permutation reasoning and list AC unification, and used it to formalize first-order classical linear logic with cut elimination.

Research Assistant, *Wharton Research Scholars*, advised by Prof. Sanjeev Khanna

Jan 2023 – Dec 2024

University of Pennsylvania, The Wharton School

- Proved pseudo-polynomial algorithms for precedence-constrained knapsack on structured poset classes – unions of three paths and one-directional bipartite posets with consecutive children – and wrote an undergraduate thesis on the work.

SELECTED TECHNICAL PROJECTS

RISC-V Pipelined Processor – *SystemVerilog, Python, FPGA*

Feb 2025 – May 2025

University of Pennsylvania, CIS 5710 Computer Organization and Design

- Designed a RISC-V pipelined processor with AXI-Lite caching, multi-cycle units, branch prediction, and memory-mapped I/O.
- Built verification testbenches, optimized LUT usage and timing behavior, and synthesized the design to FPGA to run compiled programs.

Oat Compiler – *OCaml, C, LLVM IR, x86, Clang, Dune*

Feb 2024 – May 2024

University of Pennsylvania, CIS 7000 Compilers and Interpreters

- Implemented a compiler for Oat, a C-like language with pointers and structured data, including lexing, parsing, type checking, frontend lowering, and backend code generation to LLVM IR and x86.
- Added static analyses and optimizations including mem2reg and dead-code elimination, validated against thousands of test cases.

SKILLS & RESEARCH INTERESTS

Proof assistants & verification: Lean 4, Rocq, Z3, property-based testing, static analysis.

Languages & tools: OCaml, Haskell, C/C++, Rust, Python, PyTorch, Java, SystemVerilog, LLVM IR/Clang, Docker.

Research interests: compilers, formal verification, program logics, probabilistic programming, concurrency, information flow.